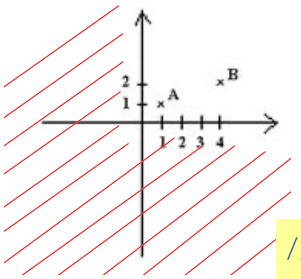
UAlg FCT
UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

Classes and objects

Readings: ( :chapt 2, 3)

1

Problem: Distance between points within the first quadrant



Write a program that, given any two points **with non-negative coordinates**, compute the integer part of the distance

```
//    What we would like for the client

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner (System.in);
        Point p;  p=new Point();
        Point a = new Point(sc.nextInt(), sc.nextInt());
        Point b = new Point(sc.nextInt(), sc.nextInt());
        System.out.println( (int) a.dist(b) );
        sc.close();
} }   System.out.println(a);
```


2

Check point: what's common in these classes?

```
class Person {
    private String name;
    public Person(String s) {
        name = s;
    }
    //…
}
```

```
class Point {
    private int x, y;
    public Point(int x, int y) {
        setX(x); setY(y);
    }
```

```
class X {
    public X() {
        //…
    }
    //…
}
```

3

Constructors

A constructor initializes the variables of the object and returns a reference to the object

```
class Point {
    private int x, y;
    public Point(int x, int y) {
        check(x); check(y);
        this.x = x;
        this.y = y;
    }
    //…
}
```

```
// Client code
Point p;                    // variable declaration
p = new Point(1, 2);  // calls the constructor
```

4

The version 1.0 of Point

```
/** @version 1.0 */

class Point {
    private int x, y;
    public Point(int x, int y) {
            check(x); check(y);
            this.x = x;
            this.y = y;
    }
    public void check(int x) {
             if (x<0) System.exit(0);
    }
    public int getX() { return x; }
    public int getY() { return y; }
}}
```

5

Violations can now be detected by the class. Where exactly?

```
public class Main {
    public static void main(String[] args) {
            Scanner sc = new Scanner (System.in);

            // Violation …
            Point a = new Point(-5, sc.nextInt()); // easily detectable here!

            Point b = new Point(sc.nextInt(), sc.nextInt());
            System.out.println( (int) a.dist(b) );
            sc.close();
    }
}
```


6

Constructor

- A constructor is called whenever a object is created, using the keyword **new**
- A constructor has the following characteristics:
  - It has the same name as the class
  - It does not have a return type (not even void!)

```
// Client code
Point p;                  // variable declaration
p = new Point(1, 2);  // calls the constructor
```

7

Default constructor

- If the class does not define any constructor, the compiler will provide a default (or non-argument) constructor.
- Default constructor allows for instance variable default initialization
- If the class define a contructor (any constructor) the compiler will not provide a default constructor.

8

Default variable initializations

- Instance variables are automatically initialized:
  - **`boolean`** types are initialized to false
  - Other primitives are initialized to the zero of their type
  - Class types are initialized to **`null`**
- It is a good programming practice to explicitly initialize instance variables within a constructor
- Local variables are ***not*** automatically initialized

9

Default constructor – example

```
// AVOID: Used for motivation only
class Point {
    private int x, y=1024;
    public int getX() {return x;}
    public int getY() {return y;}
}
```

```
public class Main {
 public static void main(String[] args) {
            Point p = new Point();
            System.out.println(p.getX());
            System.out.println(p.getY());
 }
}
```



10

Default constructor – example

```
/** @version 1.0 */

class Point {
    private int x, y;
    public Point(int x, int y) {
            check(x); check(y);
            this.x = x;
            this.y = y;
    }
    public void check(int x) {
            if (x<0) System.exit(0);
    }
    public int getX() { return x; }
    public int getY() { return y; }
}}
```

Compile time error: no default constructor available !

```
public class Main {
 public static void main(String[] args) {
            Point p = new Point();
            System.out.println(p.getX());
            System.out.println(p.getY());
 }
}
```

11

Explicit default constructor

```
/** @version 1.1 */
class Point {
    private int x, y;

    public Point() {}

    public Point(int x, int y) {
            // …
    }
    // …
}
```

12

Explicit default constructor – an improved version

```
/** @version 2.0 */
class Point {
    private int x, y;

    public Point() {
            this.x = 0;
            this.y = 0;
    }

    public Point(int x, int y) {
             // …
    }
    // …
}
```

13

Scope

- Each program element has a scope — the context where it can be referred to using its *simple* name.
- Elements can have the same name when their scope does not overlap.

```
{
   int  k = 0;
   // ...
}

for (int k = 0; k< 3; k++) {
   // OK! No problem!
}
```

14

Scope

- Each program element has a scope — the context where it can be referred to using its *simple* name.
- If the scopes of two variables overlap, the variable defined in the inner scope takes precedence over the variable with the same name in the outer scope.

```
{
  int  k = 0;
  // ...

 for (int k = 0; k< 3; k++) {
      // loop k hides outer k
  }
}
```

15

Scope

- The scope of an element is determined by the place where the element is *defined*.
- Variables with the same name in different scopes are different variables.
- Possible scopes in Java:
  - Local
  - Function
  - File
  - Class
  - Prototype
  - Package

```
class Point {
   private int x, y;
   public Point(int x, int y) {
    // x, y: the same name
    // in a different scope
   }
   // …
}
```

16

Method overloading

- Method overloading occurs when two or more methods within the *same* class have the same *name*
- To be valid, any two definitions of the method name must have different *signatures*
  - A signature consists of the name of a method together with its parameter list
  - Differing signatures must have different number and/or types of parameters

17

Method overloading

- When a name of a method is declared twice within the same class, the compiler views the second declaration as follows:

- If the return type and the signature of both methods agree, methods are viewed as a duplicate;

```
int   mult (int a, float b) { // … };
int   mult (int a, float b) { // … };   //Error – duplicate method
```

- If the signature of both methods are equal but the return type are different, the second declaration is viewed as an error;

```
float mult (int a, float b) { // … };
int   mult (int a, float b) { // … };   //Error
```

- If signatures are different in the number and/or type of parameters, both methods are viewed as *overloaded*.

```
float mult (int a,    float b) { // … };
float mult ( float a, float b) { // … };   //OK
```

18

Where are we so far?

- Java
  - First programs
  - Classes
    - Control of visibility
    - Initialization of objects: constructors
    - Method overloading